Smart Bite - Project & Architecture Overview

Comprehensive Technical Documentation, AI Architecture & System Specification

Executive Summary: *Smart Bite is an intelligent, full-stack AI-powered nutrition and fitness platform engineered to simplify diet tracking, macronutrient balance, and metabolic health. By leveraging computer vision (Google Gemini Vision), parallel model racing with automatic API key failover, and scientific MET metabolic calculation, users receive instant, highly accurate meal breakdowns, healthy alternative recommendations, and habit-forming gamification.*

1. Technology Stack

Layer	Technologies Used	Key Responsibilities
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Lucide Icons	Responsive SPA, image camera upload, interactive calendar, dynamic macro rings, dashboard visualization
Backend	Node.js, Express.js, TypeScript (ts-node)	REST API endpoints, image payload normalization, fallback model dispatch, MySQL connection pooling
Database	MySQL Server (mysql2/promise driver)	Stores user body metrics, macro goals, saved meal histories with JSON analysis, and timestamped exercise logs
AI Engine	Google Gemini 2.5/3.x SDK (@google/genai)	Visual food recognition, macro breakdown, healthy alternative generation, conversational nutrition assistant

2. Core Features & Functional Modules

A. AI Meal Scanner & Computer Vision Analysis

Users photograph or upload meals. The image is passed via base64 encoding to Google Gemini Vision models. The model identifies all visible ingredients, estimates portion weights in grams, and returns structured nutritional data: calories, protein, carbohydrates, fats, and an overall health rating.

B. Instant Healthy Alternatives Engine

Directly below the scanned image, the application displays proactive, AI-generated healthy alternatives. For example, if French fries are scanned, the engine suggests baked sweet potato wedges, explaining the glycemic and caloric benefit in an elegant badge format.

C. Real-time Net Calorie Tracking

The dashboard tracks **Net Calories = Calories Consumed - Exercise Burned**. As users log meals and workouts throughout the day, the progress bar updates in real-time against their goal.

D. Exercise & Activity Module

Allows logging of walking, running, cycling, swimming, gym workouts, and yoga. Caloric burn is calculated using standard scientific Metabolic Equivalent of Task (MET) ratings combined with user body weight and intensity: Calories Burned = (MET x 3.5 x Weight in kg x Minutes) / 200.

E. AI Meal Suggestions

Analyzes the user's remaining caloric and macronutrient budget for the day and recommends three personalized recipes with precise macros to hit their targets without exceeding limits.

F. Gamified Streaks & Global Leaderboard

Encourages consistency by awarding streak days when net calories remain within +/-15% of the user's daily target. A global leaderboard displays user rankings, all-time best streaks, and an interactive podium.

3. High-Speed AI Architecture: Parallel Racing & Key Rotation

To eliminate latency and prevent quota exhaustion (HTTP 429 errors), Smart Bite implements a cutting-edge two-tiered AI dispatch architecture in server/gemini.ts:

1. **Multi-Model Parallel Race (Promise.any):** Instead of trying fallback models sequentially (which caused 5-10s delays), the backend fires parallel requests across available models (gemini-3.6-flash, gemini-flash-latest, gemini-3.1-pro-preview, and gemini-3.1-flash-lite). The fastest successful response resolves immediately, slashing analysis times.
2. **Seamless Multi-API Key Failover Rotation:** The system scans .env for up to 5 keys (GEMINI_API_KEY, GEMINI_API_KEY1 through GEMINI_API_KEY4). If an entire model tier exhausts its quota on Key #1, the engine automatically rolls over to Key #2 and triggers a new parallel race without user interruption.

4. Relational Database Architecture (MySQL)

The database is managed via server/db.ts. On server startup, initializeDatabase() automatically creates the smartbite database and ensures all required tables and foreign keys exist.

Table	Primary Key / Foreign Key	Key Columns & Types
users	email (PK)	name (VARCHAR), goal (VARCHAR), age (INT), gender (VARCHAR), weightKg (FLOAT), heightCm (FLOAT), targetCalories (INT), targetProtein (INT), targetCarbs (INT), targetFat (INT), joinedAt (TIMESTAMP)
saved_meals	id (PK) user_email (FK -> users.email)	date (VARCHAR), time (VARCHAR), imageUrl (LONGTEXT), analysis (JSON), customNotes (TEXT)
exercises	id (PK) user_email (FK -> users.email)	date (VARCHAR), time (VARCHAR), type (VARCHAR), duration_minutes (INT), intensity (VARCHAR), calories_burned (INT), created_at (TIMESTAMP)

5. Backend REST API Endpoints

Method	Endpoint	Description
POST	`/api/analyze-food`	Uploads food image; returns macros, ingredients, alternatives via Gemini Vision
POST	`/api/suggest-meals`	Generates 3 meal ideas tailored to user's remaining daily macro deficit
POST	`/api/recommend-target`	Calculates personalized BMR/TDEE calorie and macro goals during onboarding
POST	`/api/chat`	AI health nutritionist conversational assistant for user advice and tips
GET / POST	`/api/users`	Retrieves or registers user profiles with personalized targets
GET / POST	`/api/meals`	Fetches meal history or persists newly logged meal with full analysis JSON
GET / POST	`/api/exercises`	Retrieves or saves exercise sessions with calories burned math
GET	`/api/leaderboard`	Computes rankings across all users based on longest active streaks

6. Deployment & Execution Reference

Prerequisites: Node.js (v18+) and MySQL Server. Configure root password as root1234. The server will automatically generate the smartbite database schema on start.

Environment Variables (server/.env):
GEMINI_API_KEY="primary_key"
GEMINI_API_KEY1="secondary_key"
GEMINI_API_KEY2="tertiary_key"

Execution Commands:
• Backend: cd server && npm install && npm run dev
• Frontend: cd client && npm install && npm run dev (Runs on http://localhost:5173)